

a *WalletService*. In such cases, implementing transactions based on classic ACID constraints is not practical. Though we can employ a 2-phase commit or 2PC in such cases, it leads to locking the involved databases for a longer period of time. It also hampers the ability to add and remove services over a period of time. In some cases, 2PC may not even be practical. For instance, some services may use data management techniques where the concept of committing and rolling back a transaction may not even exist. In some other cases, the participating service may be a third-party service, which we cannot lock.

The pattern named *saga* is used in such cases. To put it simply, a *saga* is a series of local transactions. Unlike in the case of 2PC, the participating services do not lock their databases beyond their local transaction. In case any local transaction that is part of a *saga* fails, the participating services run compensatory local transactions to restore consistency. In other words, the system may briefly become inconsistent during a *saga*, but eventually, the consistency is restored. For traditional data engineers, this looks odd. However, in many cases, such an eventual consistency is just good enough.

There are two popular patterns used to implement *sagas*. In the *orchestrator* pattern, an execution controller (SEC) tells each of the participating services to run a local transaction or to run a compensatory transaction. The SEC holds the current state of the *saga* at any point in time. This is really good for debugging, etc. In the *choreography* pattern, the participating services communicate among themselves to carry out the *saga*. This approach is good when the number of services is less.

To get the best results, *saga* is implemented along with two other important patterns, namely, *CQRS* and *event sourcing*.

CQRS stands for Command Query Responsibility Segregation. One set of microservices processes queries, whereas the other set processes commands like creation, updation and deletion of data. Such segregation helps in fine-tuning the respective databases for their purposes. The success of such segregation lies in how these two databases are synchronised.

If we take it a bit more forward, *event sourcing* patterns recommend not to mutate the database until it is really required. In this pattern, the command microservice only remembers the commands but does not execute them for as long as it is possible. This improves performance. When there is a need to really mutate the database, these command events are replayed.

There are a few more patterns in the area of data management like *transactional outboxing*, *API composition*, etc. Though many organisations still implement most of these patterns on their own, there are several open source frameworks and libraries like Eventuate available to aid the teams.

Deployment patterns

We already know that a service can be deployed on a dedicated box (*service instance per host*) or co-hosted on

a given box (*multiple instances per host*). We also know that services can be deployed on VMs (*service instance per VM*) or on containers (*service instance per container*). Their merits and demerits are obvious. We used the *service instance per container* pattern for deploying the UMS.

There is another alternative deployment pattern called *serverless*. This is offered by cloud providers such as AWS. In this approach, the cloud provider offers us a ready-made run-time environment. For example, it may offer JVMs. Our only job is to build the event handling functions and register them with the environment. AWS calls these functions Lambda functions. They are known as Google Functions, Azure Functions, etc, depending on the cloud provider. For instance, we may deploy an event handling function that is invoked against a specified REST API call.

For example, on the AWS platform, we can register a Lambda written in Java, Python or any other supported languages against a REST endpoint. These functions just accept the event data and return a JSON response object. Receiving the REST request, extracting the request data, invoking the registered Lambda, and rendering the JSON response are taken care of by the AWS platform. Depending on the load, AWS takes care of running the required number of Lambda instances. This way, we do not worry about building code, containerisation, orchestration, etc. We just use the API of the provider to develop and deploy the functions. Since we are no longer involved in server creation or provisioning, etc, this model is called *serverless*, though there is always a server!

Serverless is more suitable for event-driven services since the billing is based on the number of function invocations, where the function is expected to be finished in the given time.

Communication patterns

We may also refer to these as collaboration patterns. There are two predominant patterns in this area.

When services interact with each other directly by invoking them, it is said to be a *remote procedure invocation* (RPI). In this approach, a service invokes another service by calling a published REST API endpoint. It is inherently synchronous and potentially leads to an anti-pattern called a distributed monolith. This is a scenario in which all the microservices block each other by waiting for each other synchronously.

In the other case, the services may collaborate by sending messages through a broker. This is referred to as a *messaging pattern*. This approach is asynchronous, highly scalable and, at the same time, a bit complicated.

In the case of UMS, we used the Kafka message broker and that's how we want the *AddService* and *JournalService* to collaborate. Let us quickly develop the *JournalService* to prove this point.